

Tercera Pre-entrega

Este notebook continúa el trabajo realizado en la Pre-entrega 2. Se conserva la carga y la inspección inicial del dataset, y se agregan nuevos análisis y selecciones para profundizar su estructura:

- Conversión de las columnas Date y Time a tipos temporales.
- Análisis de valores atípicos en Amount mediante el criterio IQR.
- Revisión de la memoria utilizada por las columnas y comparación luego de convertir algunas variables de baja cardinalidad.
- Análisis de la cantidad de cuentas distintas y de la frecuencia con que aparecen como emisoras o receptoras.
- Aplicación de filtros para explorar operaciones internacionales de monto alto y posibles montos atípicos.
- Creación de una versión del DataFrame sin Laundering_type, para excluir esa variable de la selección preliminar de predictores y reducir el riesgo de usar información asociada a la etiqueta Is_laundering.

El objetivo de esta entrega es ampliar la inspección y el saneamiento inicial del dataset.

Ingesta, radiografía y funciones de transformación

Estructura inicial del dataset del proyecto

Proyecto: identificación de transacciones sospechosas de lavado de dinero con el dataset SAML-D.

El objetivo de este notebook es cargar el archivo, revisar su estructura y datos faltantes, y crear dos transformaciones iniciales. SAML-D es una base de datos ficticia; sus registros no representan operaciones de clientes reales.

Bloque 1 Ingesta de datos y primer vistazo

Se instalan e importan las librerías necesarias (Pandas y Gdown) y se carga el CSV en un DataFrame llamado df.

```
In [1]: %pip install pandas gdown
```

Requirement already satisfied: pandas in C:\Users\odrio\anaconda3\Lib\site-packages (3.0.3)

Requirement already satisfied: gdown in C:\Users\odrio\anaconda3\Lib\site-packages (6.4.1)

Requirement already satisfied: numpy>=2.3.3 in C:\Users\odrio\anaconda3\Lib\site-packages (from pandas) (2.4.6)

Requirement already satisfied: python-dateutil>=2.8.2 in C:\Users\odrio\anaconda3\Lib\site-packages (from pandas) (2.9.0.post0)

Requirement already satisfied: tzdata in C:\Users\odrio\anaconda3\Lib\site-packages (from pandas) (2026.2)

Requirement already satisfied: beautifulsoup4 in C:\Users\odrio\anaconda3\Lib\site-packages (from gdown) (4.15.0)

Requirement already satisfied: filelock in C:\Users\odrio\anaconda3\Lib\site-packages (from gdown) (3.29.4)

Requirement already satisfied: requests[socks] in C:\Users\odrio\anaconda3\Lib\site-packages (from gdown) (2.34.2)

Requirement already satisfied: tqdm in C:\Users\odrio\anaconda3\Lib\site-packages (from gdown) (4.68.2)

Requirement already satisfied: urllib3 in C:\Users\odrio\anaconda3\Lib\site-packages (from gdown) (2.7.0)

Requirement already satisfied: six>=1.5 in C:\Users\odrio\anaconda3\Lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

Requirement already satisfied: soupsieve>=1.6.1 in C:\Users\odrio\anaconda3\Lib\site-packages (from beautifulsoup4->gdown) (2.8.4)

Requirement already satisfied: typing-extensions>=4.0.0 in C:\Users\odrio\anaconda3\Lib\site-packages (from beautifulsoup4->gdown) (4.16.0)

Requirement already satisfied: charset-normalizer<4,>=2 in C:\Users\odrio\anaconda3\Lib\site-packages (from requests[socks]->gdown) (3.4.7)

Requirement already satisfied: idna<4,>=2.5 in C:\Users\odrio\anaconda3\Lib\site-packages (from requests[socks]->gdown) (3.18)

Requirement already satisfied: certifi>=2023.5.7 in C:\Users\odrio\anaconda3\Lib\site-packages (from requests[socks]->gdown) (2026.7.22)

Requirement already satisfied: PySocks!=1.5.7,>=1.5.6 in C:\Users\odrio\anaconda3\Lib\site-packages (from requests[socks]->gdown) (1.7.1)

Requirement already satisfied: colorama in C:\Users\odrio\anaconda3\Lib\site-packages (from tqdm->gdown) (0.4.6)

Note: you may need to restart the kernel to use updated packages.

```
In [2]: import gdown
import pandas as pd

archivo_id = "1uRs7ojeWVmAB0U5fgeUNBCAh4jqL1800"
url = f"https://drive.google.com/uc?id={archivo_id}"

gdown.download(url, "SAML-D.csv", quiet=False)

df = pd.read_csv("SAML-D.csv", low_memory=False)
display(df.head())
```

```
Downloading...
From (original): https://drive.google.com/uc?id=1uRs7ojeWVmAB0U5fgeUNBCAh4jqL1800
From (redirected): https://drive.google.com/uc?id=1uRs7ojeWVmAB0U5fgeUNBCAh4jqL1800&confirm=t&uuiid=707a6410-3ab9-4623-99d3-1554e1330b2a
To: C:\Users\odrio\Curso_CoderHouse_Data_Sciencie_I\SAML-D.csv
100%|██████████| 996M/996M [00:17<00:00, 55.8MB/s]
```

	Time	Date	Sender_account	Receiver_account	Amount	Payment_currency	Receiv
0	10:35:19	2022-10-07	8724731955	2769355426	1459.15	UK pounds	
1	10:35:20	2022-10-07	1491989064	8401255335	6019.64	UK pounds	
2	10:35:20	2022-10-07	287305149	4404767002	14328.44	UK pounds	
3	10:35:21	2022-10-07	5376652437	9600420220	11895.00	UK pounds	
4	10:35:21	2022-10-07	9614186178	3803336972	115.25	UK pounds	

Bloque 2 Radiografía técnica del dataset

shape muestra las dimensiones exactas; info() permite revisar tipos y valores no nulos; describe() resume las variables numéricas. La celda final calcula indicadores que ayudan a redactar dos observaciones basadas en los resultados observados.

```
In [3]: # Dimensiones exactas: (filas, columnas)
print(f"Dimensiones del dataset: {df.shape[0]:,} filas y {df.shape[1]} columnas")
```

Dimensiones del dataset: 9,504,852 filas y 12 columnas

```
In [4]: # Nombres de columnas y Tipos de dato
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9504852 entries, 0 to 9504851
Data columns (total 12 columns):
#   Column                                Dtype
---  -
0   Time                                  str
1   Date                                  str
2   Sender_account                       int64
3   Receiver_account                     int64
4   Amount                               float64
5   Payment_currency                     str
6   Received_currency                    str
7   Sender_bank_location                 str
8   Receiver_bank_location               str
9   Payment_type                         str
10  Is_laundering                        int64
11  Laundering_type                      str
dtypes: float64(1), int64(3), str(8)
memory usage: 1.4 GB
```

```
In [5]: # Resumen estadístico de las variables numéricas.
display(df.describe().style.format("{:,.2f}"))

q1 = df["Amount"].quantile(0.25)
q3 = df["Amount"].quantile(0.75)
iqr = q3 - q1
```

```

limite_inferior = q1 - 1.5 * iqr
limite_superior = q3 + 1.5 * iqr

cantidad_bajos = (df["Amount"] < limite_inferior).sum()
cantidad_altos = (df["Amount"] > limite_superior).sum()

total_outliers = cantidad_bajos + cantidad_altos
porcentaje_outliers = total_outliers / len(df) * 100

print(f"Límite inferior: {limite_inferior:,.2f}")
print(f"Límite superior: {limite_superior:,.2f}")
print(f"Montos por debajo del límite: {cantidad_bajos:,}")
print(f"Montos por encima del límite: {cantidad_altos:,}")
print(f"Total de posibles valores atípicos: {total_outliers:,}")
print(f"Porcentaje del total: {porcentaje_outliers:.2f}%")

```

	Sender_account	Receiver_account	Amount	Is_laundering
count	9,504,852.00	9,504,852.00	9,504,852.00	9,504,852.00
mean	5,006,619,409.97	5,006,006,469.65	8,762.97	0.00
std	2,885,813,892.13	2,884,762,983.27	25,614.95	0.03
min	9,018.00	9,018.00	3.73	0.00
25%	2,513,133,245.00	2,513,219,464.00	2,143.69	0.00
50%	5,001,016,667.00	5,002,572,491.50	6,113.72	0.00
75%	7,505,051,146.00	7,502,397,427.00	10,458.46	0.00
max	9,999,986,843.00	9,999,971,095.00	12,618,498.40	1.00

Límite inferior: -10,328.47
 Límite superior: 22,930.62
 Montos por debajo del límite: 0
 Montos por encima del límite: 429,848
 Total de posibles valores atípicos: 429,848
 Porcentaje del total: 4.52%

In [6]: *# Uso de la memoria: se verifica el uso de memoria por cada columna*

```

memoria = df.memory_usage(deep=True, index=False)

tabla_memoria = pd.DataFrame({
    "Tipo de dato": df.dtypes.astype(str),
    "Memoria (MB)": memoria / (1024 ** 2)
}).sort_values("Memoria (MB)", ascending=False)

display(tabla_memoria.style.format({"Memoria (MB)": "{:,.2f}"}))

print(f"Memoria estimada del DataFrame: {memoria.sum() / (1024 ** 2):,.2f} MB")

```

	Tipo de dato	Memoria (MB)
Laundering_type	str	221.03
Date	str	163.16
Payment_currency	str	153.86
Received_currency	str	153.55
Payment_type	str	147.64
Time	str	145.03
Receiver_bank_location	str	93.35
Sender_bank_location	str	92.00
Receiver_account	int64	72.52
Sender_account	int64	72.52
Amount	float64	72.52
Is_laundering	int64	72.52

Memoria estimada del DataFrame: 1,459.68 MB

```
In [7]: # Se convierten determinadas variables a tipo "category", "Bool" e "integer"

df["Payment_type"] = df["Payment_type"].astype("category")
df["Payment_currency"] = df["Payment_currency"].astype("category")
df["Received_currency"] = df["Received_currency"].astype("category")
df["Receiver_bank_location"] = df["Receiver_bank_location"].astype("category")
df["Sender_bank_location"] = df["Sender_bank_location"].astype("category")
df["Laundering_type"] = df["Laundering_type"].astype("category")

df["Is_laundering"] = df["Is_laundering"].astype(bool)

df["Receiver_account"] = pd.to_numeric(df["Receiver_account"], downcast = "integer")
df["Sender_account"] = pd.to_numeric(df["Sender_account"], downcast = "integer")
df["Amount"] = pd.to_numeric(df["Amount"], downcast = "float")
```

```
In [8]: # Se convierten las variables Date y Time a tipos temporales

df["Date"] = pd.to_datetime(df["Date"], format="%Y-%m-%d", errors="coerce")

df["Time"] = pd.to_timedelta(df["Time"], errors="coerce")

print(df[["Date", "Time"]].dtypes)
print("Fechas inválidas:", df["Date"].isna().sum())
print("Horas inválidas:", df["Time"].isna().sum())
```

```
Date      datetime64[us]
Time      timedelta64[us]
dtype: object
Fechas inválidas: 0
Horas inválidas: 0
```

Inicialmente, Date y Time estaban almacenadas como texto. Se convirtieron a tipos temporales para facilitar su análisis. La conversión no generó valores inválidos: ambas columnas tienen cero registros faltantes después del proceso.

```
In [9]: # Uso de la memoria luego de las transformaciones

memoria = df.memory_usage(deep=True, index=False)

tabla_memoria = pd.DataFrame({
    "Tipo de dato": df.dtypes.astype(str),
    "Memoria (MB)": memoria / (1024 ** 2)
}).sort_values("Memoria (MB)", ascending=False)

display(tabla_memoria.style.format({"Memoria (MB)": "{:,.2f}"}))

print(f"Memoria estimada del DataFrame: {memoria.sum() / (1024 ** 2):,.2f} MB")
```

	Tipo de dato	Memoria (MB)
Time	timedelta64[us]	72.52
Date	datetime64[us]	72.52
Sender_account	int64	72.52
Receiver_account	int64	72.52
Amount	float64	72.52
Laundering_type	category	9.07
Receiver_bank_location	category	9.06
Sender_bank_location	category	9.06
Received_currency	category	9.06
Payment_currency	category	9.06
Payment_type	category	9.06
Is_laundering	bool	9.06

Memoria estimada del DataFrame: 426.03 MB

```
In [10]: # Se determina la cantidad de cuentas involucradas distintas hay en las variables

for columna in ["Sender_account", "Receiver_account"]:
    frecuencias = df[columna].value_counts()

    print(f"\n{columna}")
    print(f"Cuentas distintas: {len(frecuencias):,}")
    print(f"Cuentas que aparecen más de una vez: {(frecuencias > 1).sum():,}")

    display(
        frecuencias.head(5)
        .rename("Cantidad de transacciones")
        .to_frame()
    )
```

Sender_account

Cuentas distintas: 292,715

Cuentas que aparecen más de una vez: 264,899

Cantidad de transacciones

Sender_account

4808614002	754
2938210715	753
5579295130	751
2357599526	749
3831533348	749

Receiver_account

Cuentas distintas: 652,266

Cuentas que aparecen más de una vez: 559,853

Cantidad de transacciones

Receiver_account

8600542721	751
2938210715	745
5460360634	740
5579295130	739
4808614002	737

Hallazgos de la estadística descriptiva

1. **Sender_account** y **Receiver_account** aparecen entre las variables numéricas. Sin embargo, representan identificadores de cuentas, no cantidades; por eso, sus promedios y desviaciones estándar no tienen una interpretación estadística útil.
2. **Is_laundering** está almacenada como número entero y sus valores son binarios (0 y 1). Representa la etiqueta asignada por el dataset: 1 identifica una transacción etiquetada como sospechosa y 0 una no etiquetada como sospechosa. Su media es cercana a 0, lo que muestra que los casos sospechosos son una proporción muy pequeña del total.

Bloque 3 Diagnóstico de datos faltantes

Se calcula el número y porcentaje de valores nulos por columna y se ordenan de mayor a menor porcentaje.

```
In [11]: # 1. Se calcula la cantidad de nulos y el porcentaje y se ordena de mayor a meno
nulos = df.isna().sum()
porcentaje_nulos = df.isna().mean()*100
porcentaje_nulos = porcentaje_nulos.sort_values(ascending=False)
```

2. Se arma una tabla y se ordena Las columnas por porcentaje de nulos.

```
tabla_nulos = pd.DataFrame({
    "Cantidad de Nulos": nulos,
    "Porcentaje de Nulos (%)": porcentaje_nulos
}).sort_values("Porcentaje de Nulos (%)", ascending=False)

display(tabla_nulos.style.format({"Porcentaje de Nulos (%)": "{:.4f}%"}))
```

	Cantidad de Nulos	Porcentaje de Nulos (%)
Time	0	0.0000%
Date	0	0.0000%
Sender_account	0	0.0000%
Receiver_account	0	0.0000%
Amount	0	0.0000%
Payment_currency	0	0.0000%
Received_currency	0	0.0000%
Sender_bank_location	0	0.0000%
Receiver_bank_location	0	0.0000%
Payment_type	0	0.0000%
Is_laundering	0	0.0000%
Laundering_type	0	0.0000%

Ninguna columna presenta valores nulos.

Bloque 4 Funciones personalizadas

La función `clasificar_corredor` recorre las transacciones y las clasifica según la ubicación de los bancos y las monedas de envío y recepción. Distingue ubicaciones faltantes, operaciones internacionales y operaciones nacionales con monedas iguales o distintas. Esta clasificación es descriptiva y no determina por sí sola que una transacción sea ilícita. La función `lambda` clasifica el monto de cada transacción como bajo (menor a 5.000), medio (desde 5.000 y menor a 15.000) o alto (15.000 o más). Los límites son rangos definidos para esta exploración con el objetivo de agrupar las transacciones por monto. Ambas clasificaciones se agregan al DataFrame como las columnas `Tipo_corredor` y `Categoria_monto`.

```
In [12]: def clasificar_corredor(origen, destino, moneda_envio, moneda_recepcion):

    if pd.isna(origen) or pd.isna(destino):
        return "Ubicación faltante"
    elif origen != destino:
        return "Operación Internacional"
    elif pd.isna(moneda_envio) or pd.isna(moneda_recepcion):
        return "Moneda faltante"
```

```
elif moneda_envio != moneda_recepcion:
    return "Operación Nacional, distinta moneda"
else:
    return "Operación Nacional, misma moneda"

# Columnas que se usarán, en el orden en que se pasan a la función.
columnas = [
    "Sender_bank_location",
    "Receiver_bank_location",
    "Payment_currency",
    "Received_currency"
]

resultados = []

# Recorre cada fila de esas cuatro columnas.
for fila in df[columnas].itertuples(index=False, name=None):
    resultado = clasificar_corredor(*fila)
    resultados.append(resultado)

# Se agrega la clasificación al DataFrame.
df["Tipo_corredor"] = resultados

# Lambda: asigna una categoría según el monto de la transacción.
clasificar_monto = lambda monto: (
    "Monto bajo" if monto < 5000
    else "Monto medio" if monto < 15000
    else "Monto alto"
)

df["Categoria_monto"] = df["Amount"].apply(clasificar_monto)

# Se muestran 10 filas en forma aleatoria de las columnas que nos interesan y la
display(df[columnas + ["Tipo_corredor"] + ["Categoria_monto"]].sample(10))
```

Sender_bank_location	Receiver_bank_location	Payment_currency	Received_curre
5782040	UK	UK	UK pounds
3907330	UK	UK	UK pounds
1817542	UK	UK	UK pounds
7614634	UK	UK	UK pounds
8207881	UK	UK	UK pounds
517322	UK	UK	UK pounds
8900256	UK	UK	UK pounds
1179524	UK	UK	UK pounds
317319	UK	UK	UK pounds
565458	UK	UK	UK pounds

In [13]: `# Revisa cuántas filas quedaron en cada categoría, combinando las dos columnas c`

```
display(
    df[["Tipo_corredor"] + ["Categoria_monto"]].value_counts(dropna=False)
    .rename("cantidad")
    .to_frame()
)
```

		cantidad
Operación Nacional, misma moneda	Tipo_corredor	Categoría_monto
	Monto medio	3829968
	Monto bajo	3559618
Operación Internacional	Monto alto	1025226
	Monto medio	540481
	Monto bajo	254680
Operación Nacional, distinta moneda	Monto alto	139187
	Monto medio	75651
	Monto bajo	59582
	Monto alto	20459

```
In [14]: # Resumen de cuántas filas quedaron en cada categoría de monto
resumen_montos = (
    df["Categoría_monto"]
    .value_counts()
    .rename("Cantidad")
    .to_frame()
)

resumen_montos["Porcentaje (%)"] = (
    resumen_montos["Cantidad"] / len(df) * 100
)

display(resumen_montos.style.format({"Porcentaje (%)": "{:.2f}%"}))
```

	Cantidad	Porcentaje (%)
Categoría_monto		
Monto medio	4446100	46.78%
Monto bajo	3873880	40.76%
Monto alto	1184872	12.47%

Estos rangos se utilizan para describir los montos y no representan umbrales oficiales ni determinan que una transacción sea sospechosa.

```
In [15]: # Ver uso de memoria de esas dos variables creadas

memoria = df[["Tipo_corredor", "Categoría_monto"]].memory_usage(
    deep=True,
    index=False
)

tabla_memoria = pd.DataFrame({
    "Tipo de dato": df[["Tipo_corredor", "Categoría_monto"]].dtypes.astype(str),
    "Memoria (MB)": memoria / (1024 ** 2)
})
```

```
display(tabla_memoria.style.format({"Memoria (MB)": "{:,.2f}"}))

print(f"Memoria estimada de esas columnas: {memoria.sum() / (1024 ** 2):,.2f} MB
```

	Tipo de dato	Memoria (MB)
Tipo_corredor	str	364.07
Categoria_monto	str	167.40

Memoria estimada de esas columnas: 531.47 MB

In [16]: *# Se convierten las variables creadas a tipo "category"*

```
df["Tipo_corredor"] = df["Tipo_corredor"].astype("category")
df["Categoria_monto"] = df["Categoria_monto"].astype("category")
```

In [17]: *# Ver uso de memoria de esas dos variables creadas luego de haberlas convertido*

```
memoria = df[["Tipo_corredor", "Categoria_monto"]].memory_usage(
    deep=True,
    index=False
)

tabla_memoria = pd.DataFrame({
    "Tipo de dato": df[["Tipo_corredor", "Categoria_monto"]].dtypes.astype(str),
    "Memoria (MB)": memoria / (1024 ** 2)
})

display(tabla_memoria.style.format({"Memoria (MB)": "{:,.2f}"}))

print(f"Memoria estimada de esas columnas: {memoria.sum() / (1024 ** 2):,.2f} MB
```

	Tipo de dato	Memoria (MB)
Tipo_corredor	category	9.06
Categoria_monto	category	9.06

Memoria estimada de esas columnas: 18.13 MB

Bloque 5 Saneamiento y Selección

Se crea un subconjunto de datos mediante un filtro por aquellas transacciones internacionales y de monto alto

```
In [18]: filtro_internacionales = df[
    (df["Tipo_corredor"] == "Operación Internacional")
    & (df["Categoria_monto"] == "Monto alto")
]

print("Operaciones internacionales de monto alto:",
      f"{len(filtro_internacionales):,}")

display(filtro_internacionales.head(10))
```

Operaciones internacionales de monto alto: 139,187

	Time	Date	Sender_account	Receiver_account	Amount	Payment_currency	Reci
120	0 days 10:39:50	2022- 10-07	5931838828	8407065812	16594.14	UK pounds	
194	0 days 10:41:48	2022- 10-07	3712660061	2534644942	23161.44	UK pounds	
267	0 days 10:44:23	2022- 10-07	5263982631	7080357663	27479.44	Yen	
339	0 days 10:47:33	2022- 10-07	8389148416	7489694854	23582.14	UK pounds	
340	0 days 10:47:34	2022- 10-07	4121324384	9281561360	22158.31	Euro	
454	0 days 10:51:34	2022- 10-07	4121324384	8777101435	18207.77	Euro	
481	0 days 10:53:04	2022- 10-07	6127913747	8384869861	18066.71	UK pounds	
675	0 days 11:00:59	2022- 10-07	7877700106	2534644942	25336.71	UK pounds	
796	0 days 11:05:13	2022- 10-07	9880098011	7892884492	16738.96	UK pounds	
881	0 days 11:07:45	2022- 10-07	5263982631	7080357663	27492.73	Yen	



In [19]: *# Se crea un subconjunto de datos mediante un filtro por aquellas transacciones*

```
filtro_outliers_altos = df[df["Amount"] > limite_superior]

print("Montos atípicos altos:", f"{len(filtro_outliers_altos):,}")
display(filtro_outliers_altos.head(5))
```

Montos atípicos altos: 429,848

	Time	Date	Sender_account	Receiver_account	Amount	Payment_currency	Reci
42	0 days 10:36:36	2022- 10-07	1445991735	8941775586	33322.12	UK pounds	
44	0 days 10:36:45	2022- 10-07	109525315	8518184956	38707.49	UK pounds	
128	0 days 10:40:08	2022- 10-07	7501327339	9329185542	29600.19	UK pounds	
138	0 days 10:40:20	2022- 10-07	7470940363	8095203071	24267.62	UK pounds	
170	0 days 10:41:06	2022- 10-07	4390278719	8172080174	25562.20	UK pounds	

```
In [20]: # Se elimina la columna de tipología ya que es una columna que se intentara predecir
df_sin_tipologia = df.drop("Laundering_type", axis = 1)

print("Dimensiones del DataFrame original:", df.shape)
print("Dimensiones sin Laundering_type:", df_sin_tipologia.shape)
```

Dimensiones del DataFrame original: (9504852, 14)

Dimensiones sin Laundering_type: (9504852, 13)

```
In [21]: ## Bloque 6 Reflexión
```

En la inspección inicial no se encontraron valores nulos. Sender_account y Receiver_account están almacenadas como números, pero representan identificadores de cuentas, por lo que sus valores no deben interpretarse como cantidades. Sin embargo, son útiles para agrupar transacciones y analizar cuántas operaciones envía o recibe cada cuenta, así como sus relaciones con otras cuentas. Is_laundering contiene las etiquetas binarias del dataset ficticio. La columna Amount presenta una gran diferencia entre los montos habituales y los valores máximos. Por eso se utilizó el criterio IQR para identificar posibles valores atípicos altos. Estos valores son inusuales desde el punto de vista estadístico, pero eso no significa por sí solo que sean sospechosos. Para el proyecto resultan relevantes el monto, el tipo de pago, las monedas, las ubicaciones bancarias, el momento de la transacción y las cuentas involucradas. Is_laundering se considera la etiqueta objetivo. La columna Laundering_type se excluye de la vista preliminar.